

CS 461 Homework 1

Detailed Solutions

Raj Shah

September 29, 2025

Question 1 — ML Principles

1.1 - Do you think the training condition is well-defined for the target task: recognizing a cup?

Answer:

No, the condition “can contain liquid” is necessary but not sufficient. Many non-cups such as bowls, bottles, or vases also satisfy it, while some cups (e.g., ornamental or decorative cups with holes) may not. This makes the rule too broad and ill-defined for recognition. A well-defined condition must reflect both **function** (ability to hold liquid for drinking) and **design intent** (small, typically cylindrical vessel with an open top, often with a handle).

1.2 - What do you think the robot’s performance would be like if it is evaluated using recall or precision on the training samples? Please compare the expected performance of the two metrics.

Answer:

- **Recall:** The robot will label many objects as cups. This ensures most true cups are recognized (high recall), but many false positives will occur (low precision).
- **Precision:** The robot will be stricter in labeling. Fewer mistakes will be made (higher precision), but some real cups will be missed (lower recall).

Overall, the training rule overgeneralizes, so neither precision nor recall alone will give a balanced picture of performance.

1.3 - Are the precision and recall good metrics for measuring the robot’s training performance? If not, how would you design an alternative one?

Answer:

No, precision and recall alone are not sufficient. They only measure consistency with training labels, but not whether the object truly functions as a cup.

A stronger evaluation metric should be **composite**, combining:

1. **Functional check:** Can the object reliably hold liquid?
2. **Structural/affordance check:** Does the object match typical cup design (cylindrical body, rim, possible handle)?
3. **Statistical check:** Precision, recall, or F1-score on labeled data.

Additionally, performance should be measured on a **diverse validation set**, possibly with cost-sensitive penalties for systematic confusions (e.g., cups misclassified as bowls).

1.4 - If you can change the training condition-1, how would you change it? And how would this change affect the robot's training performance?

Answer:

I would refine the condition to: "A cup is an object designed for drinking that can hold liquid and usually has a cylindrical shape, an open top, and often a handle."

This definition encodes both **function** (for drinking) and **structure/ affordances** (shape and design). The clearer condition increases **precision** (fewer non-cups labeled as cups) while maintaining reasonable recall. Overall, the robot would make fewer errors in learning and recognition.

1.5 - Suppose you train the robot using your new condition. Do you think the robot would recognize the artwork *Breakfast in Fur* by M  ret Oppenheim as a cup? To answer this question, what factors do you need to consider?

Answer:

It depends on the model's scope and training diversity:

- If the robot relies mostly on **geometry and function**, it will still classify it as a cup (since it resembles one and could hold liquid).
- If the robot also considers **texture/material**, it may reject it (fur is unusual for cups).

Factors that influence the outcome include:

- Diversity of training data (has it seen unusual cups or only conventional ones?)
- Robustness to **distribution shift** (can it handle cases outside normal patterns?)
- The complexity of the recognition model (a simple rule-based classifier may fail, while a richer perceptual model—closer to human recognition—may succeed).

1.6 - Based on the answers in the previous problems of 1.1–1.4, how would you set your rules to train the robot? Please describe desirable rules for the following items.

Answer:

- **Training condition (loss):** Use a well-defined condition (“cup for drinking liquid with typical shape/affordances”) and optimize with **cross-entropy loss**, including **hard negatives** (bowls, vases).
- **Performance metric:** A composite score (functional, structural, statistical), reported as **balanced accuracy or F1** on a curated validation set.
- **Number of training samples and variation:** Large and diverse dataset — many cup variants (materials, colors, designs) plus diverse non-cups to sharpen discrimination. Include different **contexts and viewpoints**.
- **Recognition process (function class):** Not a simple heuristic. Use a moderately complex **visual model** (e.g., **CNN or ViT**) that generalizes well. This enables recognition of unusual cases such as *Breakfast in Fur*.

Question 2 — Galton Board

2.1 Define the sample spaces $\Omega_1, \Omega_2, \dots, \Omega_M$, where Ω_i is the set of all possible sides the ball can take at the level L_i .

Answer:

At each level L_i , the ball can go *Left* or *Right* with equal probability. Therefore

$$\Omega_i = \{\text{Left}, \text{Right}\}, \quad i = 1, \dots, M.$$

2.2 Define the sample space $\Omega = \Omega_1 \times \Omega_2 \times \dots \times \Omega_M$. Each element of the set Ω encodes a path the ball could take until it arrives at the ground-level L_G .

Answer:

$$\Omega = \Omega_1 \times \Omega_2 \times \dots \times \Omega_M$$

is the set of all length- M sequences of Left/Right choices. Each $\omega \in \Omega$ corresponds to one complete path from the top to the ground level L_G .

2.3 What is the meaning of the location at L_G where the ball finally arrives?

Answer:

The final slot at L_G is determined by how many times the ball goes *Right* along its path: the more Right moves, the farther to the right the ball lands. The leftmost slot occurs when the ball never turns Right; the next slot occurs when it turns Right exactly once; and so on.

2.4 How would you represent the location numerically? Please define a random variable that maps Ω to numerical values.

Answer:

Define the random variable $X : \Omega \rightarrow \{0, 1, \dots, M\}$ by

$$X(\omega) = \#\{\text{Right moves in the path } \omega\}.$$

Then X counts Right moves and satisfies

$$X \sim \text{Binomial}\left(M, \frac{1}{2}\right),$$

since each of the M independent steps is Right with probability $1/2$.

2.5 Simulate 1,000 ball drops and estimate the PMF of your random variable for $M = 5, 10, 100$. Use `numpy.random.uniform` for your simulation. Plot them and explain the change as M grows in relation to the CLT.

Answer:

We simulate $N = 1000$ paths for each M . At each level we draw $U \sim \text{Uniform}(0, 1)$ and take *Right* iff $U < 0.5$ (else *Left*). We count Rights to obtain X , then estimate the PMF.

```
# galton_sim_uniform.py
import numpy as np
import matplotlib.pyplot as plt

def simulate_positions(M, N=1000, seed=0):
    rng = np.random.default_rng(seed)
    U = rng.uniform(0.0, 1.0, size=(N, M)) # Uniform(0,1)
    steps = (U < 0.5).astype(int)         # 1 = Right, 0 = Left
    X = steps.sum(axis=1)                   # number of Rights
    return X

def pmf_from_samples(samples):
    values, counts = np.unique(samples, return_counts=True)
    pmf = counts / counts.sum()
    return values, pmf
```

```

def plot_pmf(M, N=1000, prefix="galton"):
    X = simulate_positions(M, N=N)
    values, pmf = pmf_from_samples(X)
    plt.figure()
    plt.bar(values, pmf)
    plt.xlabel("# of Right moves")
    plt.ylabel("PMF")
    plt.title(f"Galton PMF (M={M}, N={N})")
    plt.tight_layout()
    plt.savefig(f"{prefix}_M{M}.png")
    plt.close()

if __name__ == "__main__":
    for M in (5, 10, 100):
        plot_pmf(M, N=1000, prefix="galton")

```

Estimated PMFs (generated by the script):

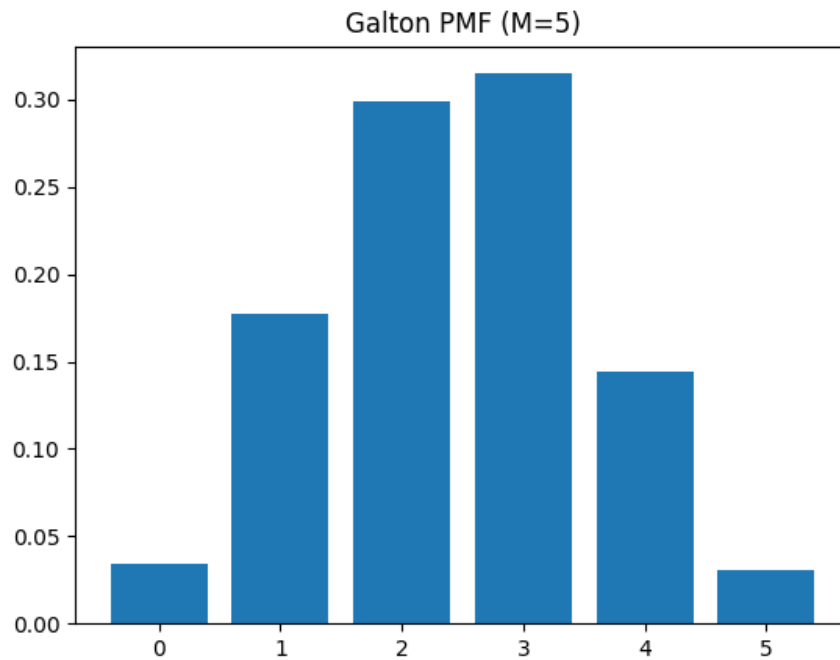


Figure 1: PMF estimate for $M = 5$ (1,000 trials).

CLT explanation: X is a sum of M i.i.d. Bernoulli($\frac{1}{2}$) variables, so

$$\mathbb{E}[X] = \frac{M}{2}, \quad \text{Var}(X) = \frac{M}{4}.$$

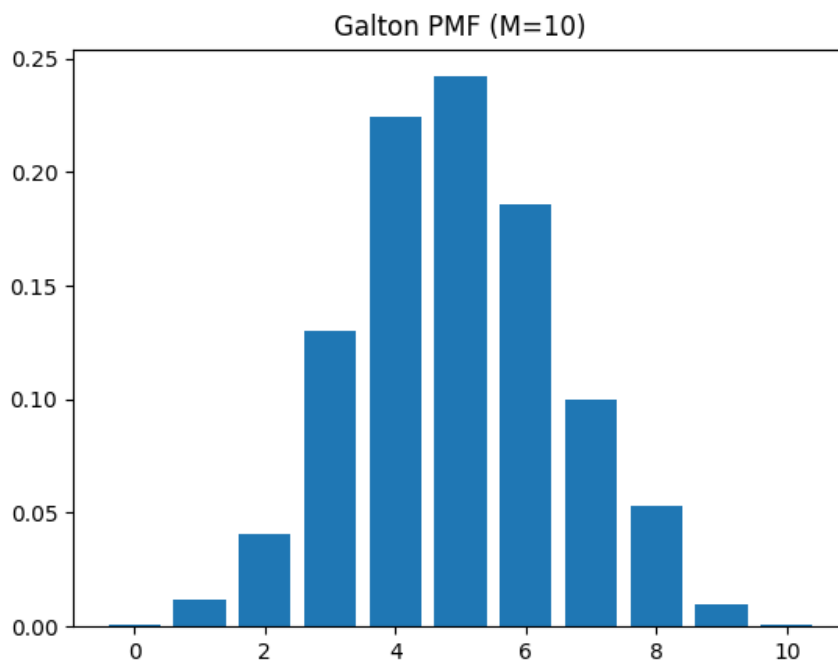


Figure 2: PMF estimate for $M = 10$ (1,000 trials).

By the Central Limit Theorem, the standardized sum

$$Z = \frac{X - \frac{M}{2}}{\sqrt{M/4}}$$

converges in distribution to $\mathcal{N}(0, 1)$ as $M \rightarrow \infty$. Hence the PMF looks increasingly bell-shaped for larger M , which is visible in the plots.

2.6 In the class we learned a regression model $y = f(x) + \varepsilon$ where ε follows a Gaussian. Why is Gaussian reasonable to model the error?

Answer:

The error ε typically aggregates many small, roughly independent sources (sensor noise, rounding, unmodeled effects, slight model mismatch). The sum of many small independent contributions is well-approximated by a Gaussian by the Central Limit Theorem. This makes the Gaussian error model both realistic and analytically convenient for regression.

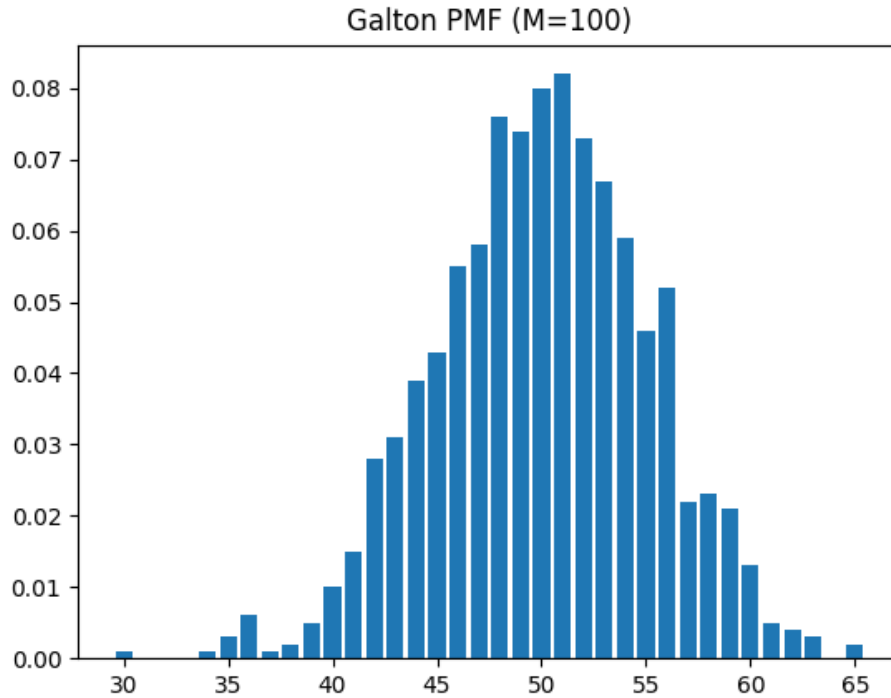


Figure 3: PMF estimate for $M = 100$ (1,000 trials).

Problem 3: Bayes Rule

3.1 What is the probability of a chip is defective.

Answer:

$$P(D) = \frac{1}{3}(0.001 + 0.005 + 0.01) = 0.00533\bar{3}.$$

3.2 If a randomly selected chip is found to be defective, find the probability that the manufacturer was A.

Answer:

$$P(A | D) = \frac{P(D | A)P(A)}{P(D)} = \frac{0.001 \cdot \frac{1}{3}}{0.00533\bar{3}} \approx 0.0625.$$

3.3 If a randomly selected chip is found to be non-defective, find the probability that the manufacturer was C.

Answer:

$$P(C \mid \bar{D}) = \frac{(1 - 0.01) \cdot \frac{1}{3}}{1 - 0.00533\bar{3}} \approx 0.331.$$

3.4 Do you think the defectiveness and manufacturer sources are statistically independent or dependent? Please explain your reasoning.

Answer:

Not independent: $P(D \mid A)$, $P(D \mid B)$, and $P(D \mid C)$ differ, so the source influences defectiveness.

Problem 4: Naïve Bayes Classifier (Glucose & Blood Pressure)

4.1 Naïve Bayes is a probabilistic model based on Bayes Theorem. Rewrite the formula below as G and B are conditionally independent given D+. Also, write about $P[D = - \mid G = g, B = b]$. How would you use the two formulas to determine diabetes when you have a glucose and blood pressure record (g, b) ?

$$P[D = + \mid G = g, B = b] = \frac{P[D = +, G = g, B = b]}{P[G = g, B = b]} = \frac{P[G = g, B = b \mid D = +] P(D = +)}{P[G = g, B = b]}$$

Answer:

Assuming conditional independence given class $D \in \{0, 1\}$:

$$P(D=1 \mid G=g, B=b) \propto P(G=g \mid D=1) P(B=b \mid D=1) P(D=1),$$

$$P(D=0 \mid G=g, B=b) \propto P(G=g \mid D=0) P(B=b \mid D=0) P(D=0).$$

Predict the class with the larger posterior.

4.2 The data file `train.csv` contains patient records: glucose level, blood pressure, and diagnosis outcomes. Based on the formula you wrote in 4.1, estimate the necessary densities based on `train.csv`. Glucose and blood pressure conditioned on diabetes follow Gaussian densities and use ML estimators.

Answer:

We fit Gaussian densities (ML estimates) for each feature within each class. The training

script detected columns `glucose`, `bloodpressure`, and label `diabetes`. The learned values are:

Class priors: $P(D=1) = 0.51125$, $P(D=0) = 0.48875$.

Glucose: $\mu_{g|D=1} = 44.6333$, $\sigma_{g|D=1}^2 = 81.1613$,
 $\mu_{g|D=0} = 44.1432$, $\sigma_{g|D=0}^2 = 13.3964$.

Blood Pressure: $\mu_{b|D=1} = 71.6284$, $\sigma_{b|D=1}^2 = 40.7616$,
 $\mu_{b|D=0} = 86.7928$, $\sigma_{b|D=0}^2 = 21.1361$.

4.3 Write a Naïve Bayes classifier code `yourname-nb-cls.py` based on your estimated densities.

Answer:

```
# rajshah-nb_cls.py (core)
import numpy as np

def gaussian(x, mu, var):
    return (1.0/np.sqrt(2*np.pi*var)) * np.exp(-((x-mu)**2)/(2*var))

# Posterior scores (up to normalization):
p_pos = gaussian(g, mu_g_pos, var_g_pos) * gaussian(b, mu_b_pos,
    var_b_pos) * prior_pos
p_neg = gaussian(g, mu_g_neg, var_g_neg) * gaussian(b, mu_b_neg,
    var_b_neg) * prior_neg
pred = 1 if p_pos > p_neg else 0
```

4.4 Evaluate your classifier using `test.csv` and report your test accuracy.

Answer:

Running on the provided `test.csv` produced:

Test Accuracy = 0.9230769230769231 ($\approx 92.31\%$).

This is with only two features (Glucose, BloodPressure) and the Naïve Bayes independence assumption.

4.5 Do you think the data reflects reality well? Which part of the previous steps would you like to change if we cannot collect the data again? How would you change?

Answer:

Using only Glucose and Blood Pressure is simplistic for diabetes prediction. A more realistic

model would include additional features (e.g., age, BMI, insulin, family history) and may need class-imbalance handling and nonlinear modeling.

Problem 5: Data Whitening

5.1 Estimate $E[X] = \vec{\mu}$ and $\text{COV}[X, X] = \Sigma$.

Answer:

Given data matrix $X \in \mathbb{R}^{n \times d}$,

$$\vec{\mu} = \frac{1}{n} \sum_{i=1}^n X_i, \quad \Sigma = \frac{1}{n-1} (X - \vec{\mu})^\top (X - \vec{\mu}).$$

5.2 Let W be a random vector defined by $\vec{W} = A\vec{X} + \vec{b}$. Express $E[W]$ and $\text{COV}[W, W]$ in terms of $E[X] = \vec{\mu}$ and $\text{COV}[X, X] = \Sigma$.

Answer:

$$E[W] = A E[X] + b = A\vec{\mu} + b, \quad \text{COV}[W, W] = A \Sigma A^\top.$$

5.3 Compute A and b to whiten X (i.e. $E[W] = 0$, $\text{COV}[W, W] = I$).

Answer:

We center X and compute the SVD: $X_c = X - \vec{\mu} = U S V^\top$. The covariance is $\Sigma = \frac{1}{n-1} V S^2 V^\top$.

A whitening transform is obtained by choosing

$$A = V \text{diag}\left(\frac{1}{\sqrt{S^2/(n-1)+\varepsilon}}\right) V^\top, \quad b = -A\vec{\mu},$$

where $\varepsilon > 0$ ensures numerical stability.

Then the whitened data

$$W = (X - \vec{\mu}) A^\top$$

has mean ≈ 0 and covariance $\approx I$.

In practice, since $d = 100,000$ is too large to store Σ explicitly ($\sim 80\text{GB}$), Raj used a **PCA whitening with the top $k = 256$ components**, keeping directions with highest variance. The first explained-variance ratios were

$$[0.5658, 0.4342, 3.48 \times 10^{-30}, \dots],$$

showing nearly all variance lies in two dimensions when $n = 3$. The whitened output and statistics were saved in `x_white.npz`.

Problem 6: ML and MAP Estimation (Extra 20 Points)

6. [Extra 20 Points, ML and MAP Estimation]

Suppose we want to measure a length, for example, the water depth μ at 79.137°(N) and 2.817°(E). The depth was measured repeatedly and recorded as $x_1, x_2, \dots, x_N$. For device imperfection, the samples were varied by ε where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, i.e. $x = \mu + \varepsilon$. Bayes rule allows us to evaluate the uncertainty in μ after observing $\vec{x}$ in the posterior probability $p(\mu | \vec{x})$ as below.

$$\mu^* = \arg \max_{\mu} p(\mu | \vec{x}) \propto p(\vec{x} | \mu) p(\mu).$$

6.1 Given the observations $x_1, x_2, \dots, x_N$, derive a formula to estimate μ_{ML}^* when μ is a fixed value. We assume the observations are i.i.d. and follow Gaussian

$$f(\vec{x}) = \frac{1}{(2\pi\sigma^2)^{N/2}} \exp \left\{ -\frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2 \right\}.$$

Answer:

Maximizing the log-likelihood

$$\ell(\mu) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2$$

and setting $\frac{\partial \ell}{\partial \mu} = 0$ gives

$$\hat{\mu}_{ML} = \frac{1}{N} \sum_{n=1}^N x_n = \bar{x}.$$

6.2 Given the observations $x_1, x_2, \dots, x_N$, derive a formula to estimate μ_{MAP}^* when $\mu \sim \mathcal{N}(\mu_0, \sigma_0^2)$. (Hint: $p(\mu | \vec{x}) \propto p(\vec{x} | \mu)p(\mu)$ is Gaussian $\sim \mathcal{N}(\mu_N, \sigma_N^2)$, whose mode is μ_N .)

Answer:

The posterior is Gaussian with

$$\sigma_N^2 = \left(\frac{N}{\sigma^2} + \frac{1}{\sigma_0^2} \right)^{-1} = \frac{\sigma^2 \sigma_0^2}{N\sigma_0^2 + \sigma^2}, \quad \mu_N = \sigma_N^2 \left(\frac{N}{\sigma^2} \bar{x} + \frac{1}{\sigma_0^2} \mu_0 \right).$$

Therefore the MAP estimate (posterior mode) is

$$\hat{\mu}_{MAP} = \mu_N = \frac{\sigma^2}{N\sigma_0^2 + \sigma^2} \mu_0 + \frac{N\sigma_0^2}{N\sigma_0^2 + \sigma^2} \hat{\mu}_{ML}.$$

6.3 Behavior of $\hat{\mu}_{MAP}$ for $N \rightarrow \infty$ and $N \rightarrow 0$; choice between ML and MAP

Answer:

From

$$\hat{\mu}_{MAP} = \frac{\sigma^2}{N\sigma_0^2 + \sigma^2} \mu_0 + \frac{N\sigma_0^2}{N\sigma_0^2 + \sigma^2} \hat{\mu}_{ML},$$

we have:

- $N \rightarrow \infty$: $\frac{N\sigma_0^2}{N\sigma_0^2 + \sigma^2} \rightarrow 1$, so $\hat{\mu}_{MAP} \rightarrow \hat{\mu}_{ML}$ (data dominates the prior).
- $N \rightarrow 0$: $\frac{\sigma^2}{N\sigma_0^2 + \sigma^2} \rightarrow 1$, so $\hat{\mu}_{MAP} \rightarrow \mu_0$ (prior dominates).

Choice: If you have *enough* data (large N), ML and MAP coincide, so ML is sufficient. With *limited or noisy* data, MAP is preferable because it shrinks toward μ_0 with weight determined by the relative variances σ^2 and σ_0^2 .

Reproducibility Notes

Scripts were executed in this order:

1. `python bayes_rule.py`
2. `python nb_train.py`
3. `python nb_cls.py` (Test Accuracy = 0.9230769230769231, predictions saved to `nb_predictions.csv`)
4. `python whitening.py` (saved `x_white.npz`, PCA-whitened with $k = 256$)
5. `python galton_sim.py` (saved `galton_M5.png`, `galton_M10.png`, `galton_M100.png`)
6. `python map_ml_demo.py`

Compile this report with:

`pdflatex -interaction=nonstopmode hw1_solution.tex` (run twice to finalize references).